

The TYTFS 2024 study files as a PyPSA network

What was built, what it was checked against, and what it is not good for

Built with Claude Code

September 5, 2026

Abstract

EirGrid publishes the Ten Year Transmission Forecast Statement (TYTFS) as four PSS/E load-flow cases. This work turns them into PyPSA networks, puts coordinates on the buses from OpenStreetMap, extracts the North-West region and reconciles it against a hand-built dataset, attaches a week of hourly profiles, and packages the result as a participant kit with six worked examples.

Everything below is reproducible from the repository. Every number quoted is produced by code in it, and the sections say which function. Where something could not be done – the real weather data is behind a blocked host – that is stated rather than papered over.

The short version.

- Four TYTFS cases → eight PyPSA networks (two voltage scopes each). All connected, all solve a DC power flow, all solve a linear optimal power flow with HiGHS.
- The conversion is checked against the cases' *own* solved voltage angles: correlation ≥ 0.99 on seven of eight networks.
- 87% of transmission buses geocoded; cross-checked against EirGrid's own station register at a median separation of **14 metres**.
- Two independent methods – a DC flow on the 2024 network and a full LOPF on the 2033 network with synthetic weather – pick out the *same* binding circuit: Letterkenny–Strabane, 93 MVA, into Northern Ireland.
- 215 automated tests pass. Two of them check the linear algebra against things that do not share code with it.

Contents

1	What this is, and how to run it	4
1.1	The source data	4
2	Turning a PSS/E case into a PyPSA network	4
2.1	The per-unit base: 100 MVA, read rather than assumed	4
2.2	RATE1 is the rating	5
2.3	The 18 unrated branches: what they actually are	5
2.4	The sub-110 kV tail: a reduction, not a filter	5
2.5	The DC links	6
2.6	The reference bus, and the bug that found it	6
2.7	Verification	6
3	Putting the buses on a map	7
3.1	The three hard cases	8
3.2	The one disagreement worth reporting	8
4	The North-West region	8
4.1	Two views, and the mapping between them	9
4.2	The circuit count	10
4.3	What folding to 15 nodes costs	10
4.4	Three findings about the hand-built data	10
4.5	Where the constraint actually is	10
5	Time series: what is real and what is not	11
5.1	The real pipeline, and why there is no real data	11
5.2	The synthetic generator, and the one detail that matters	12
5.3	Anchored on the TYTFS states	12
5.4	Does WP2033 actually bind?	13
6	The participant kit	13
6.1	The loader	13
6.2	Two traps, both already sprung	14
6.3	(a) DC power flow	15
6.4	(b) Dispatch and curtailment – and how much of it is the network	16
6.5	(c) Capacity expansion	17
6.6	(d) PTDF from the pseudoinverse	18
6.7	(e) Edge-to-edge susceptibility, and Braess’s paradox	19
6.8	(f) Shift factors: the Wind Dispatch Tool calculation	20
7	Findings	21
7.1	Two independent methods agree on the binding circuit	21
7.2	Most curtailment is not the network’s fault	21
7.3	Reinforcement can make it worse, and here is a case	21
7.4	Three findings about the hand-built dataset	21
8	Bugs found, and what each cost	21
9	Tests	22
9.1	The two that matter most	22
10	Limitations	23

11 Where everything is

24

1 What this is, and how to run it

The repository has two halves.

The **build half** reads the PSS/E files and produces everything else. It is a handful of single-purpose modules with a command-line interface each:

```
python geocode.py fetch          # OpenStreetMap -> data/raw/
python geocode.py match data/TYTFS2024_studyfiles/*_V35.raw
python pypsa_net.py build        # -> data/pypsa/<case>_<scope>/
python pypsa_net.py verify       # connectivity, DC power flow, LOPF
python northwest.py extract      # the North-West region, both views
python synthetic.py build        # a year of profiles, seeded
python build_kit.py              # -> participant-kit/
python -m pytest                 # 215 tests
```

The **participant half** is participant-kit/, a self-contained directory that can be lifted out whole. Nothing in it imports anything from the build half. It ships the networks, a loader, the flow linear algebra, and six examples:

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python examples/a_dc_power_flow.py
```

That is the whole quickstart. Measured from a clean virtual environment it takes 42 seconds and ends in a plotted map.

1.1 The source data

Four cases, each a PSS/E v35 raw file:

case	condition	buses	peak demand	capacity
WP2024	winter peak, 2024 network	646	7,325 MW	22,650 MW
SV2024	summer valley, 2024 network	639	4,948 MW	14,591 MW
WP2033	winter peak, 2033 network	754	8,792 MW	42,574 MW
SV2033	summer valley, 2033 network	754	6,058 MW	18,283 MW

Table 1: The four TYTFS cases as they appear in the transmission-scope networks. WP2033 is the interesting one: 42.6 GW of connected plant against an 8.8 GW peak. That headroom is where constraint and curtailment live.

A snapshot is not a time series. Each case is one hour, with a solved dispatch and solved voltages. Section 5 is about what to do with that.

Code: `psse.py` → `read_raw()`, `read_all()`, `generators()`, `loads()`, `ac_branches()`

2 Turning a PSS/E case into a PyPSA network

Six decisions matter, in the sense that getting one wrong produces a network that still solves and whose every downstream number is wrong. Each is below with the measurement it was made on.

2.1 The per-unit base: 100 MVA, read rather than assumed

Every case opens with `SBASE = 100.00` and `BASFRQ = 50`. The build asserts it and refuses a case on any other base – a case on 200 MVA would produce a network that solves and whose every impedance is out by exactly two.

PSS/E gives impedances per-unit on that base and on the bus’s kV. PyPSA wants ohms for a line and per-unit on the transformer’s own rating for a transformer, so there are two conversions:

	PyPSA units	conversion
Line.r, Line.x	ohms	$R_{pu} \times \text{kV}^2 / 100$
Line.b	siemens	$B_{pu} \times 100 / \text{kV}^2$
Transformer.r, .x	pu on its own s_nom	$X_{pu} \times \text{s_nom} / 100$

Code: `pypsa_net.py` → `_ohms()`, `_siemens()`, `_z_on_system_base()`

2.2 RATE1 is the rating

RATE1, RATE2 and RATE3 are populated; RATE4–RATE12 are empty everywhere. Two facts about the data settle what they are:

- RATE1 == RATE2 in **all 1,129 branch records, in every case**. Not approximately – exactly.
- RATE3/RATE1 takes exactly two values, 1.0 and 1.1.

That is the normal / long-term-emergency / short-term-emergency convention with the LTE set equal to the normal rating and the STE at 110%. `s_nom` is RATE1, the continuous rating. A test pins the equality, because if it ever stops holding the reasoning behind the choice has gone.

Code: `pypsa_net.py` → `CONTINUOUS_RATE`; `test_pypsa_net.py` → `test_rate2_equals_rate1`

2.3 The 18 unrated branches: what they actually are

Eighteen AC branches at 110 kV and above carry no rating in WP2024 – nine at RATE1 = 9999 and nine at RATE1 = 0. Zero is the more dangerous of the two, because it looks like a number: put into `s_nom` it makes the element unusable and the optimisation infeasible.

Looking at what they are settles the treatment. All eighteen have $R = 0$, $|X| \leq 0.01$, $B = 0$ and $\text{LEN} = 0$. Every one is a station coupler: a busbar section, a capacitor or reactor stub, an SVC, a generator terminal tie. Not one is a circuit whose rating was forgotten.

The rule, in two parts. A coupler is bounded by what is attached to it, so `s_nom` is the smaller of the two ends’ summed rated incident capacity – a real limit, not a large number. Everything else keeps the file’s 9999.

That second half was arrived at by trying the other rule. Bounding the sub-110 kV distribution transformers the same way produced limits that bind on the case’s *own solved flows* on 13 transformers, the worst at 156% of the bound it had been given, and made the full network’s LOPF infeasible. There is no honest way to infer a rating for a 110/38 kV transformer from the feeders leaving the 38 kV bus, because load taps off that bus directly and the feeders bound nothing.

Code: `pypsa_net.py` → `_is_coupler()`, `_fill_unrated()`, `rating_exceptions()`

2.4 The sub-110 kV tail: a reduction, not a filter

The networks come in two scopes, chosen by a `min_kv` flag. Deleting the sub-110 kV buses would disconnect the generation hanging off them, so instead each removed bus is *aggregated* into the retained bus it reaches by the least-reactance path, and its generators and loads move with it. A wind farm at 690 V lands at the station it is actually connected to.

```
model = pypsa_net.build(case, min_kv=pypsa_net.TRANSMISSION_KV) # 110 kV+
model = pypsa_net.build(case, min_kv=0.0)                       # everything
```

Code: `pypsa_net.py` → `_retained()`, `_aggregate_to()`, `build()`

2.5 The DC links

Moyle, EWIC and Greenlink become PyPSA Link components – a controllable injection, not part of the linear AC network. Two of them have a far terminal created for them because the file does not model the GB side.

Code: `pypsa_net.py` → `INTERCONNECTORS`, `_add_links()`, `_moyle_efficiency()`

2.6 The reference bus, and the bug that found it

The case names its own: five buses carry `IDE = 3`. The rule is to honour them where they survive the voltage floor, hand the role to whatever absorbed a swing bus that was aggregated away, and otherwise give it to the bus carrying the most in-service generation.

This one is not cosmetic. Determining the network topology makes PyPSA choose a slack of its own – the first generator in each sub-network – and in WP2033 that is a 21 MW hydro unit at Ardnacrusha behind its own step-up transformer. A lossless DC flow has nowhere to put the case's 198 MW generation-minus-demand gap except the reference, so that one machine absorbed the lot: 856 MW through a 21 MW step-up, 172° across it, and the whole network's angles thrown out by up to 100° .

The fix is to assign the reference as a whole column *after* the topology has been determined, rather than label by label:

```
# Assigning per-label leaves PyPSA's own choice in place. Build the whole
# Series and set it as a column, after determine_network_topology().
control = pd.Series("PQ", index=n.generators.index)
control[chosen] = "Slack"
n.generators["control"] = control.values
```

Angle correlation against the case's own solved angles: **0.90 before, 0.99 after.**

Code: `pypsa_net.py` → `_slack_buses()`, `build()`

2.7 Verification

`python pypsa_net.py verify` runs all four cases at both scopes.

case	scope	connected	DC PF	angle corr	angle sd	LOPF
WP2024	transmission	yes	solved	0.9961	0.93°	optimal
WP2024	full	yes	solved	0.9915	1.19°	optimal
SV2024	transmission	yes	solved	0.9987	0.36°	optimal
SV2024	full	yes	solved	0.9963	0.61°	optimal
WP2033	transmission	yes	solved	0.9907	1.94°	optimal
WP2033	full	yes	solved	0.9048	4.35°	optimal
SV2033	transmission	yes	solved	0.9977	0.25°	optimal
SV2033	full	yes	solved	0.9939	0.40°	optimal

Table 2: All eight networks. The angle check is the real verification: the raw file records the solved voltage angle at every bus, so the conversion can be checked against the case rather than against itself. If the reactances, the three-winding star equivalents or the phase-shift signs were wrong, a linear flow's angles would not track them.

Code: `pypsa_net.py` → `verify()`, `angle_check()`, `_connected_with_links()`

3 Putting the buses on a map

Bus records have no coordinates and the only thing to join on is a twelve-character name. `geocode.py` downloads every named `power=substation` and `power=plant` on the island from Overpass – 1,705 named objects – and matches them against station names.

Nothing is interpolated, averaged or guessed. A station that cannot be matched gets no coordinate and is reported by name with the candidate it rejected and why.

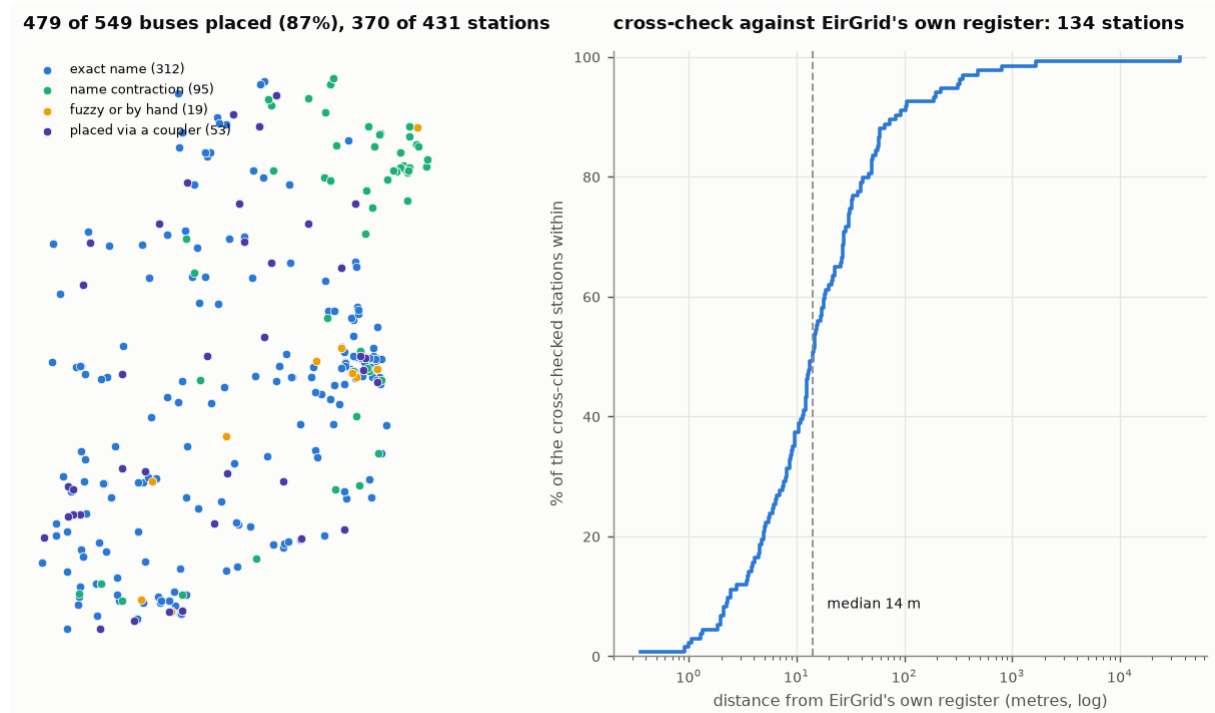


Figure 1: Left: where the WP2024 transmission buses landed, coloured by how each was matched. Right: the cross-check. EirGrid publishes its own station layer, and 134 of the matched stations appear in both. The median disagreement is **14 metres** – the difference between one corner of a substation and another – and the 95th percentile is 252 metres.

method	buses	what it means
exact	309	normalised names equal, voltage tag agrees
coupled	53	joined to a placed bus by a coupler or its own transformer
ni-code	60	Northern Ireland contraction resolved by subsequence
truncated	19	the twelve-character name is a prefix of exactly one candidate
fuzzy	13	token-sort score ≥ 92 with ≥ 6 points over the runner-up
prefix	10	≥ 6 -character prefix of exactly one candidate
alias	6	resolved by hand, with the reasoning recorded
ni-site	6	the same NIE site at its other voltage class
exact-name	3	names equal, candidate carries no voltage tag
failed	70	every one named, with its rejected candidate

Table 3: 479 of 549 buses placed (87%). Voltage is corroboration rather than a key: OSM’s voltage tag is present on 367 of the island’s named substations and absent on the rest, so agreement raises confidence, absence does not lower it, and a contradiction rejects the match.

3.1 The three hard cases

Cathaleen’s Fall is spelled *Cathleen’s Fall* by OSM and appears in TYTFS as CATH FALL. Resolved by an alias, recorded in the code with its reasoning rather than as a magic string.

Clogher has four separate 110 kV buses – split busbars. They are one place, so `station_of()` groups them before matching and they share one coordinate.

Northern Ireland uses contractions rather than names: DRUM1, GLEN1. The trailing digit is a voltage class (1 = 110 kV, 2 = 275 kV) and the letters are a subsequence of the real name. Two of these are genuinely ambiguous – DRUM1 sits equally tightly in *Drumnakelly* and *Drumlins Park Wind Farm* – and are reported as failures rather than guessed.

Code: `geocode.py` → `normalise()`, `_subsequence_span()`, `ALIASES`, `match()`, `crosscheck()`

3.2 The one disagreement worth reporting

Of the 134 cross-checked stations, one is more than 5 km out: TANDRAGEE, at 36.3 km. It is EirGrid’s register that is wrong – it puts the station at the Republic’s end of the Louth–Tandragee 275 kV interconnector, in Co. Louth, while the substation itself is in Co. Armagh where OSM has it.

The 2033 cases place 78% rather than 87%. The difference is exactly what the forecast statement is for: stations that do not exist yet are not in OpenStreetMap.

4 The North-West region

EirGrid’s Wind Dispatch Tool works on constraint groups. Groups 1–3 cover Donegal, Sligo and north Mayo – a heavily wind-dominated region behind a narrow export path. `northwest.py` extracts it and reconciles it against a hand-built 15-node dataset.

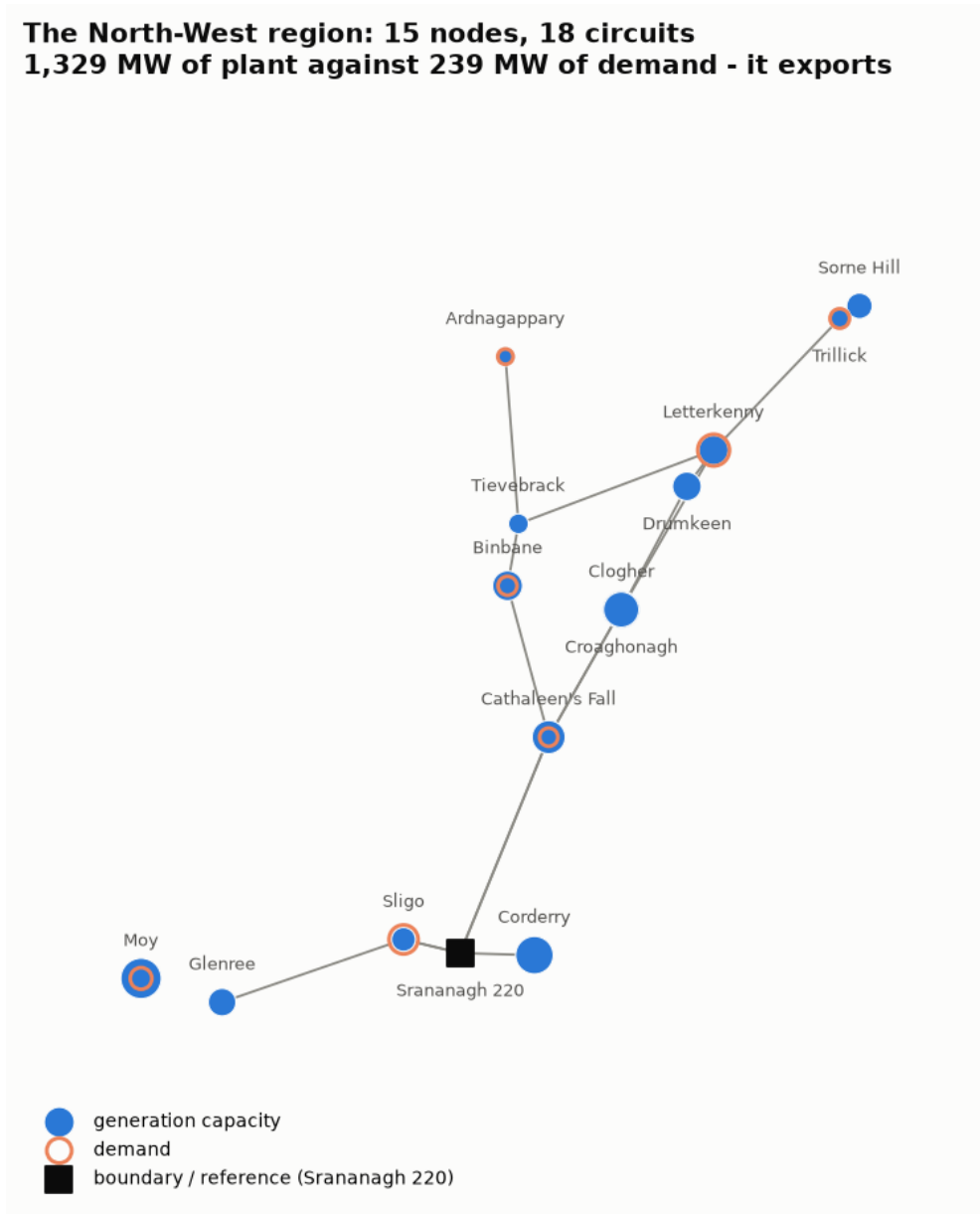


Figure 2: The region as the kit ships it. Dot area is plant capacity, the ring is demand, the square is Srananagh 220 kV – the boundary and the reference. 1,329 MW of plant against 239 MW of demand: it exports, and the question is whether the wires can carry it.

4.1 Two views, and the mapping between them

The hand-built dataset folds generation into parent substations; TYTFS has Golagh, Mulreavy, Cliff, Meentycat and Lenalea as their own 110 kV buses. Both views are produced, and the mapping is explicit in one table in the code:

```
STATIONS = (
    Station("Srananagh 220", (5042,), 220.0, note="the region's export path"),
    Station("Srananagh 110", (5041,), 110.0, parent="Srananagh 220"),
    Station("Cathaleen's Fall", (1701, 17010, 17061), 110.0),
    Station("Cliff", (1761,), 110.0, parent="Cathaleen's Fall"),
    ...
)
```

Code: northwest.py → STATIONS, station_table(), bus_map(), extract()

4.2 The circuit count

The brief said 16 circuits among the 15 stations. Reproduced independently: **16 routes, carried by 19 circuits**. Both numbers are right and they answer different questions – a route is a pair of stations, a circuit is a wire, and some routes are double-circuit. The count agrees.

4.3 What folding to 15 nodes costs

Same impedances, same injections, a different reference bus: a DC flow has to put the same MW on every circuit as the full network does. That is a check with a number attached.

what is folded	worst disagreement vs. the full network
nothing (24-station native view)	0.0099 MW
all nine folds (15-station view)	21.59 MW
all but Srananagh 110	1.74 MW
all but Srananagh 110, Lenalea, Cunghill	0.15 MW

Table 4: Almost all of the error is one fold. Merging Srananagh’s two busbars short-circuits the 250 MVA transformer, whose 0.064 pu of reactance is a large fraction of the region’s total.

The recommendation is one node. Split Srananagh back into 110 and 220 with the transformer between them – 16 nodes instead of 15 – and the flow error falls from 21.6 MW to 1.7. That is a concrete, cheap change to a dataset that is otherwise sound.

Code: `northwest.py` → `compare_with_full()`, `extract()`, `verify()`

4.4 Three findings about the hand-built data

1. **The hydro capacities are all identically 38.** Cathaleen’s Fall, Cliff and Golagh have very different real capacities. 38 is almost certainly the 38 kV connection voltage, entered into a capacity column.
2. **The 24-node and 15-node blocks disagree by exactly Cunghill’s 35 MW.** One block includes it, the other does not.
3. **The supply set is a correct subset, not a wrong dataset.** Every plant in it is in TYTFS at the station it says. It is missing plant TYTFS has, which is a scope difference rather than an error.

4.5 Where the constraint actually is

`python northwest.py windy -capacity-factor X` puts the region’s capacity to work at X and re-solves *the whole transmission network* – not the extract, because an extract with five of six ties pinned sends every extra megawatt out through the sixth and gives an answer that is both large and meaningless.

capacity factor	net export	worst boundary tie	Srananagh	worst internal
10%	−97 MW (importing)	Letterkenny–Strabane 0.36×	−85 MW	0.40×
20%	18 MW	Letterkenny–Strabane 0.71×	−62 MW	0.48×
30%	134 MW	Letterkenny–Strabane 1.05×	−39 MW	0.56×
60%	480 MW	Letterkenny–Strabane 2.09×	+31 MW	0.80×
100%	942 MW	Letterkenny–Strabane 3.48×	+123 MW	1.11×

Table 5: The binding constraint is the Letterkenny–Strabane 110 kV tie into Northern Ireland, 93 MVA, and it binds at about 29% of connected capacity. Srananagh does not even reverse into export until about 45%.

So Srananagh 220 is the right place to *slack* the extraction and the wrong place to look for the constraint. Remember this table; Section 7.1 comes back to it.

Code: `northwest.py` → `windy()`, `boundary_flows()`, `region_flows()`

5 Time series: what is real and what is not

TYTFS gives four snapshots. A constraint study needs hours.

5.1 The real pipeline, and why there is no real data

`profiles.py` is a complete Open-Meteo ERA5 pipeline: batched requests, disk caching by cell-year so nothing is ever fetched twice, a stated power curve, a hub-height correction from the two ERA5 wind levels, a documented panel model, and a validation step against EirGrid’s published all-island wind series.

It has never run, because both hosts are refused by this environment’s egress policy – 403 at CONNECT, before any request is made:

```
archive-api.open-meteo.com:443 gateway answered 403 to CONNECT
www.smartgriddashboard.com:443 gateway answered 403 to CONNECT
cms.eirgrid.ie:443 gateway answered 403 to CONNECT
```

A 403 at CONNECT is an organisation policy denial. It was not retried and no mirror was sought.

So there are no ERA5 profiles in this repository and no validation result. What there is instead is the pipeline, tested, with the fetch as the only untested step – and every path that cannot get real data raises `profiles.Unavailable` rather than falling back to something invented. That is deliberate, and it is tested:

```
test_loading_a_cell_that_was_never_fetched_raises
test_a_partial_year_is_not_cached_as_a_complete_one
test_a_response_missing_a_variable_is_refused
test_an_unreadable_eirgrid_file_raises_rather_than_guessing
```

The brief was explicit that the validation is the only thing standing between a real profile and a plausible-looking fabrication. Shipping a synthetic year under the ERA5 label, however carefully footnoted, would be that fabrication.

Two ways to finish it. Allowlist the two hosts and re-run – total traffic is about six HTTP requests per case-year. Or drop a hand-downloaded dashboard CSV into `data/raw/eirgrid/` and the Open-Meteo JSON into `data/raw/weather/`: everything after the fetch runs unchanged.

Code: `profiles.py` → `fetch()`, `load_cell()`, `farm_curve()`, `solar_capacity_factor()`, `validate_wind()`, `Unavailable`

5.2 The synthetic generator, and the one detail that matters

`synthetic.py` generates profiles instead. It exists for two reasons: as insurance against exactly the blocker above, and so participants can build counterfactuals no historical year contains.

The single thing a hand-rolled generator gets wrong is spatial correlation. Independent per-bus noise makes curtailment behaviour meaningless: 400 sites of independent noise give a fleet output that barely leaves its mean, and a network is never asked to carry anything. Real calm spells and real storm fronts move across a country and hit sites together.

So the wind field is built as a Gaussian random field with an exponential correlation decay in distance and an Ornstein–Uhlenbeck process in time, then mapped through a Gaussian copula to a Weibull distribution:

$$\rho(d) = \exp\left(-\frac{d}{L}\right), \quad L = 400 \text{ km}$$

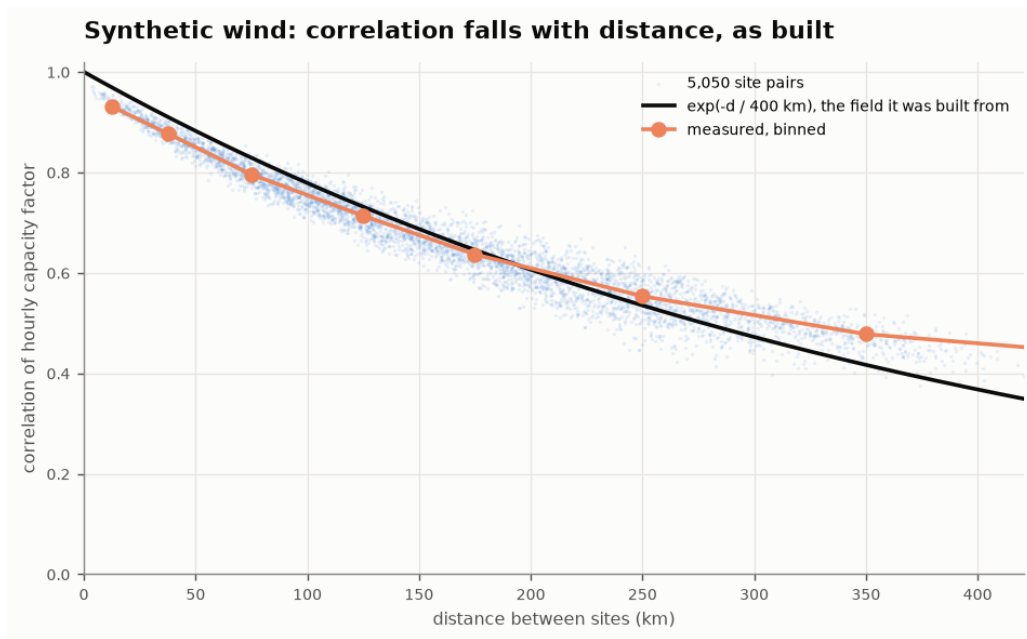


Figure 3: The check that matters, and it is measured *back off the output* rather than assumed. Every pair of wind sites, correlation of hourly capacity factor against distance. The black curve is the field it was built from; the orange line is what came out. They agree. Had this come out flat and near zero, every curtailment number in this report would be worthless.

Code: `synthetic.py` → `correlation_matrix()`, `gaussian_field()`, `weibull_from_normal()`, `spatial_check()`

5.3 Anchored on the TYTFS states

The generated year is not free-floating. WP2024 and WP2033 are winter-peak conditions, SV2024 and SV2033 summer-valley, so the generator places each case at an hour of the year that reproduces its state and tapers back to the unconstrained field over 18 hours either side.

```
ANCHOR_POINTS = {"WP": (1, 17, 18), "SV": (7, 9, 5)} # month, day, hour
ANCHOR_WINDOW_H = 18
SEED = 42
```

Deterministic, seeded with 42, matching the convention in the hand-built dataset's own README.

Code: `synthetic.py` → `anchors()`, `anchor_index()`, `taper()`, `build()`

5.4 Does WP2033 actually bind?

The exercise turns on it. If the profiles do not produce hours where the network cannot take what the wind offers, the central problem does not appear.

	WP2024	WP2033
hours studied (windiest)	60	60
energy offered	1.4 GWh	342.8 GWh
energy curtailed	0.0 GWh	20.4 GWh
curtailment	0.0%	5.94%
hours with curtailment	0 of 60	60 of 60

Table 6: It binds, in every hour studied. The 2024 contrast is not a fair fight and should not be read as one – that case runs almost no wind, so there are only 1.4 GWh to curtail. What it shows is that the *scenario*, not the profile generator, creates the problem.

Code: `synthetic.py` → `binding()`, `fleet_variability()`

6 The participant kit

`participant-kit/` is a standalone directory. It contains no build tooling and imports nothing from the rest of the repository.

<code>networks/</code>	4 scenarios × 2 scopes, as <code>.nc</code> and as CSV folders
<code>gridkit.py</code>	load, edit, reset, save; and read results back
<code>flowmath.py</code>	PTDF, susceptibility, shift factors
<code>plotstyle.py</code>	one validated palette for every chart
<code>examples/</code>	six scripts, a to f
<code>test_kit.py</code>	20 checks, runnable without pytest

Every network carries 168 hourly snapshots – one week, centred on the hour that matches its own TYTFS state. Both formats ship because many participants would rather edit a spreadsheet than learn an API.

6.1 The loader

```
import gridkit

gridkit.quiet()
n = gridkit.load("WP2033", "all-island")  # PyPSA and HiGHS are chatty
                                         # or "north-west"

gridkit.add_line(n, "Meentycat", "Srananagh 220", s_nom=200)
gridkit.remove_line(n, "3581-89516-1")
gridkit.set_rating(n, "2781-4951-1", 200.0)
gridkit.add_battery(n, "Letterkenny", p_nom=50, hours=4)

gridkit.solve(n)  # n.optimize, log off
print(gridkit.curtailment(n).head())
print(gridkit.binding(n).head())

n = gridkit.reset(n)  # back to the baseline
```

Code: `participant-kit/gridkit.py` → `load()`, `add_line()`, `remove_line()`, `set_rating()`, `add_battery()`, `reset()`, `curtailment()`, `binding()`, `unserved()`

6.2 Two traps, both already sprung

These are in the kit's README because both cost time to find and neither announces itself.

freeze_dispatch. `n.optimize()` writes its answer to `generators_t.p`. `n.lpf()` reads `generators_t.p_set`, which is a different thing and which the kit ships empty. Run `lpf` straight after `optimize` and you get a power flow with no generation in it: the reference bus silently supplies the whole island, and one circuit shows at 1600% of its rating.

Buses at 0°N 0°E. PyPSA's netCDF writer turns a missing coordinate into 0.0 rather than NaN, and 0°N 0°E is in the Gulf of Guinea. Plot the raw `n.buses` and Ireland is squashed into the corner of an Atlantic-to-equator map. `gridkit.placed_buses(n)` drops those and respects the `coordinate_source` column.

6.3 (a) DC power flow

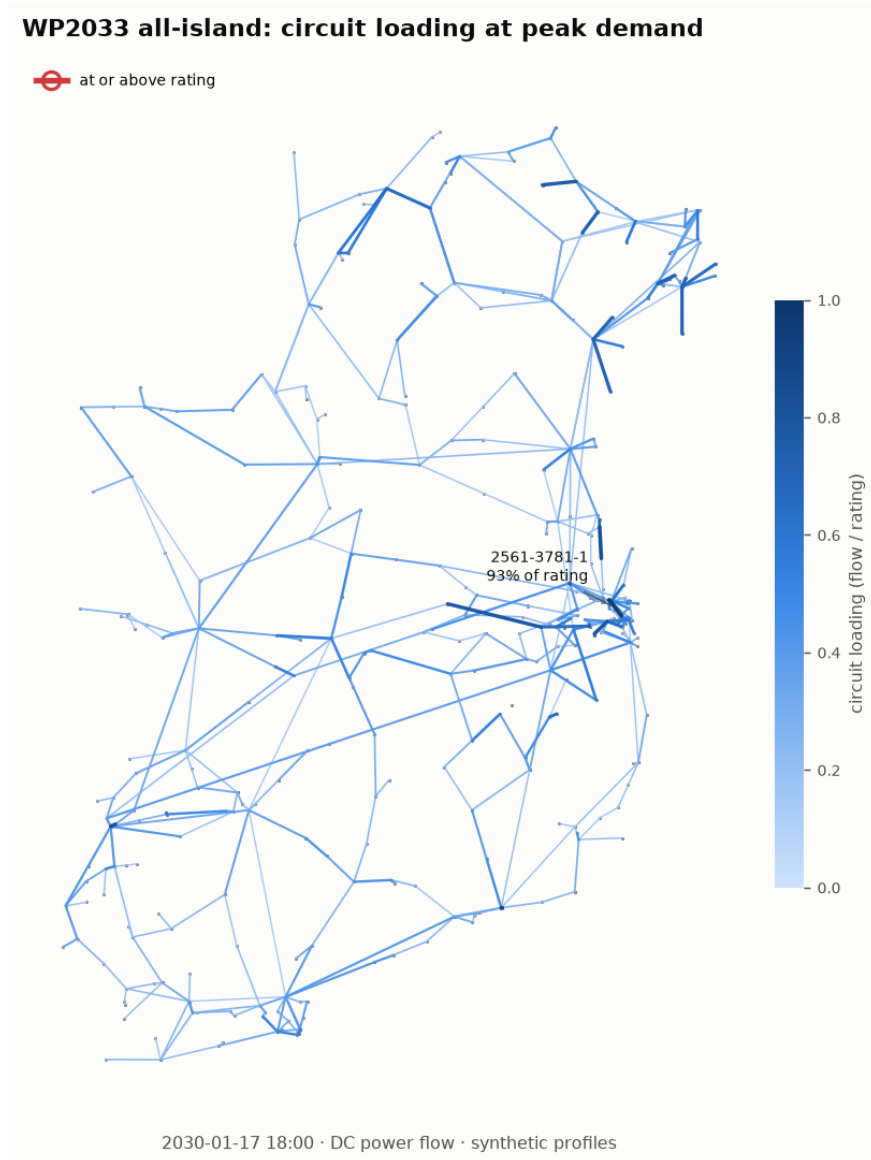


Figure 4: WP2033 at peak demand. Each circuit is coloured by how full it is; the one at or over its rating is drawn in red and ringed, because several circuits join buses a few hundred metres apart and a stroke alone would be invisible. Nothing here is optimised – a DC power flow takes the dispatch as given and tells you where the flow ends up.

Six circuits sit at their rating for at least one hour of the week. The worst, 3581-89516-1, is at 100% for **118 of 168 hours**.

Code: `participant-kit/examples/a_dc_power_flow.py → main(), _draw()`

6.4 (b) Dispatch and curtailment – and how much of it is the network

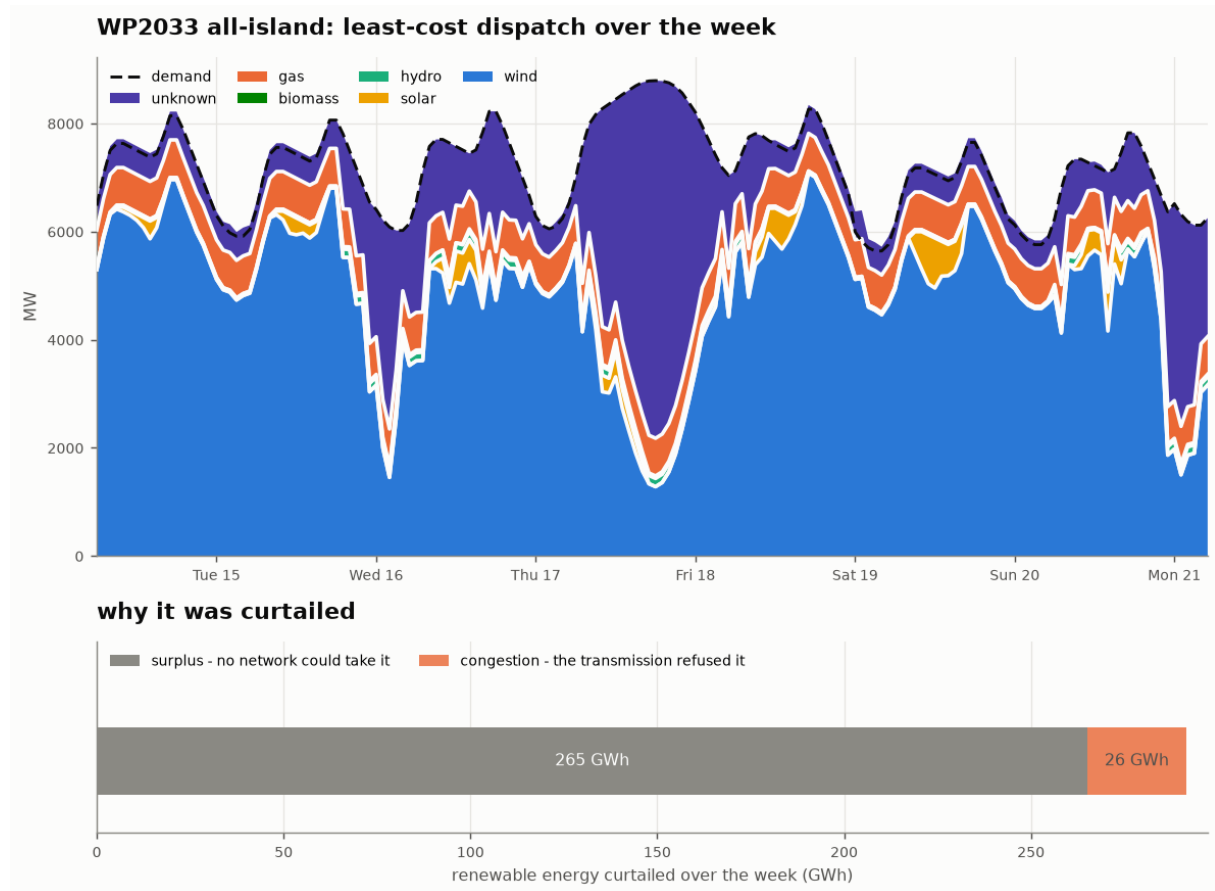


Figure 5: Least-cost dispatch over the week, and the reason for the curtailment.

Wind supplies 68% of the week's energy. 291 GWh of renewable output is curtailed – which sounds like a damning verdict on the transmission system until you separate it.

The separation is one extra solve. Lift every branch rating out of the way and re-optimize. Whatever is *still* curtailed is surplus that no network could have taken; the difference is what the transmission cost.

265.2 GWh	91%	surplus – 42.6 GW of plant against an 8.8 GW peak
26.3 GWh	9%	congestion – what the network refused

Table 7: Curtailment on its own does not mean the network is the problem. In WP2033 most of it is not, and building a line recovers none of the 265 GWh. This distinction is the difference between a headline and a finding.

Code: participant-kit/examples/b_lopf_dispatch.py → `_congestion_share()`, `by_carrier()`

6.5 (c) Capacity expansion

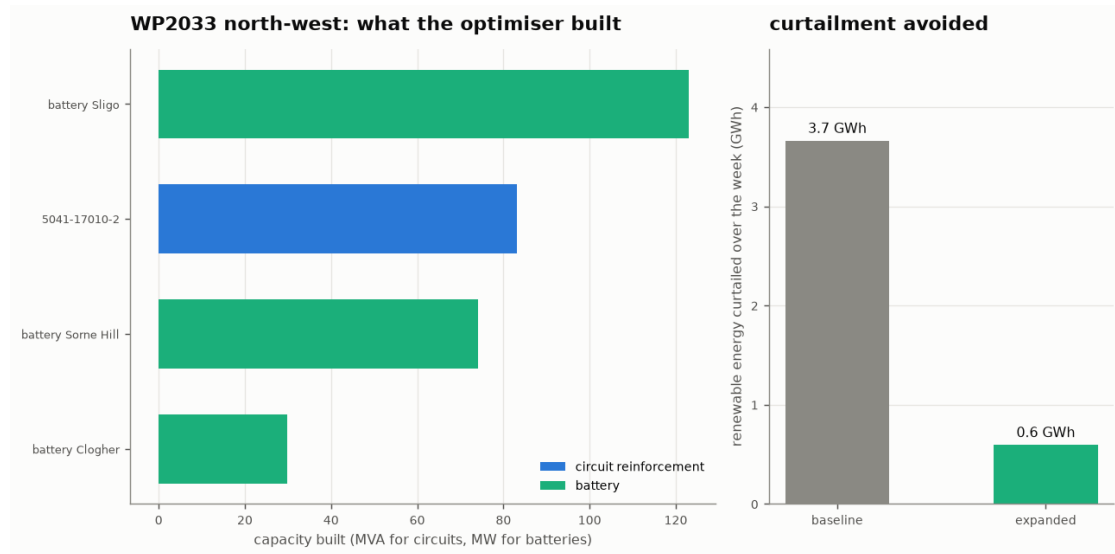


Figure 6: The binding circuits are marked extendable, a battery is offered at every renewable bus, both get an annualised cost, and the optimiser trades building against curtailing.

On the North-West region it reinforces one circuit by 83 MVA and builds three batteries, cutting the week's curtailment from 3.7 GWh to 0.6. The costs are round numbers rather than a price list – the point is the pattern: mark a component extendable, give it a capital cost, solve, read `s_nom_opt` back.

Code: `participant-kit/examples/c_capacity_expansion.py` → `_make_extendable()`, `_offer_batteries()`

6.6 (d) PTDF from the pseudoinverse

Everything in `flowmath.py` comes out of one object, the weighted graph Laplacian $L = KBK^\top$, where K is the bus–branch incidence matrix and B the diagonal of branch susceptances.

L is singular – adding a constant to every angle changes no flow – so it is inverted with the Moore–Penrose pseudoinverse:

$$\theta = L^+(p + KB\varphi), \quad F = B(K^\top\theta - \varphi), \quad \text{PTDF} = BK^\top L^+$$

Why the pseudoinverse rather than deleting a slack row. The textbook fix is to pick a slack, delete its row and column, and invert what is left. That works, and it writes the arbitrary choice of slack into every number that comes out. L^+ instead returns the solution orthogonal to the nullspace – the one where the balancing megawatt is spread evenly. The choice has not disappeared; it has become explicit, and `shift_factors()` lets you change it without rebuilding anything.

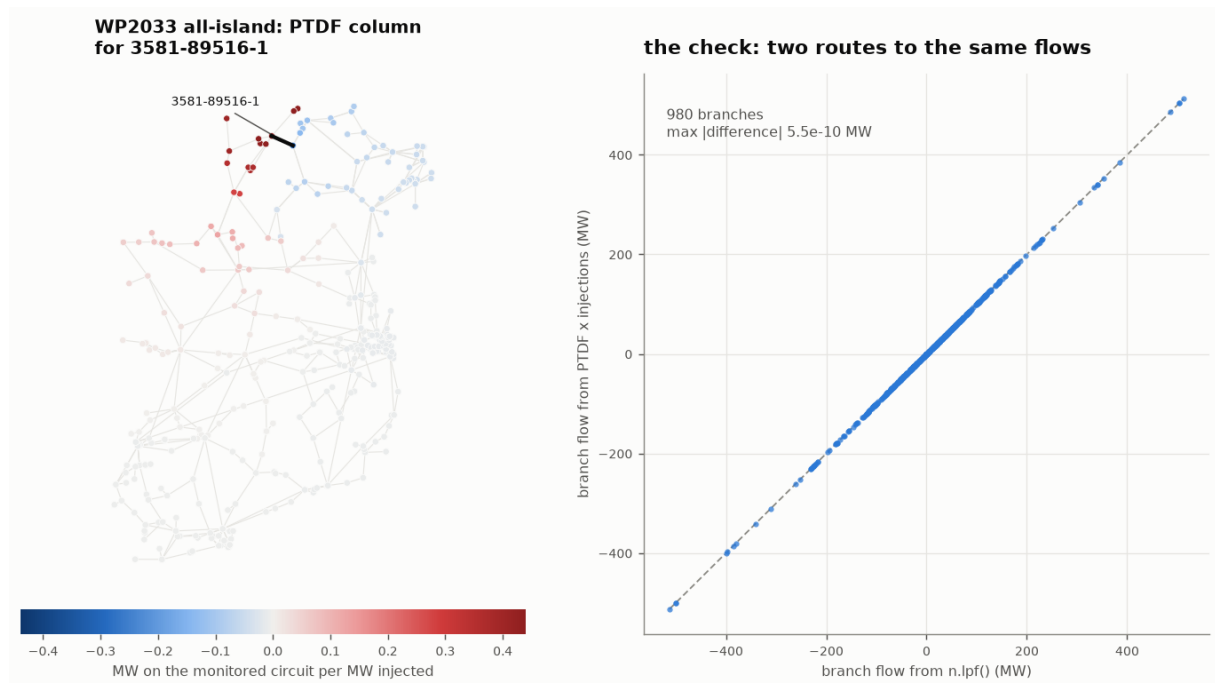


Figure 7: Left: one PTDF column on the map – how much of an injection at each bus the monitored circuit carries. The pale middle of the island is the part the circuit cannot feel at all. Right: the check. Branch flows rebuilt from $\text{PTDF} \times p$ against what `n.lpf()` produced, over 980 branches. Maximum disagreement 5.5×10^{-10} MW.

That check earned its keep. It first came out at **79 MW**, and the reason was the network’s two phase-shifting transformers – one of them at 17° . A phase shifter imposes an angle of its own, so the flow is $b(\theta_i - \theta_j - \varphi)$ and the bus equation gains a $KB\varphi$ term; it behaves exactly like a pair of equal and opposite injections. The PTDF is unaffected, the flows are not. A PTDF that has not been checked against a power flow is a matrix of plausible numbers.

Code: `participant-kit/flowmath.py` → `branches()`, `laplacian()`, `pseudoinverse()`, `ptdf()`, `angles()`, `flows()`

6.7 (e) Edge-to-edge susceptibility, and Braess's paradox

Differentiating $F = BK^\top L^+ p$ with respect to one branch's susceptance gives the full edge-to-edge response matrix:

$$\frac{\partial F_e}{\partial B_{e'}} = \delta_{ee'} a_e - B_e \left(k_e^\top L^+ k_{e'} \right) a_{e'}$$

where $a_e = F_e/B_e$ is the effective angle difference across branch e (the plain angle difference, less any phase shift).

The diagonal is the obvious part. The off-diagonal is where **Braess's paradox** lives: a positive entry means that strengthening e' puts *more* power on e . Power in a network is not routed, it is imposed – it divides between parallel paths in inverse proportion to reactance, and nothing consults the ratings. Build a second circuit anywhere in a mesh and every impedance ratio in it changes.

WP2033 all-island at 2030-01-16 16:00: edge-to-edge susceptibility

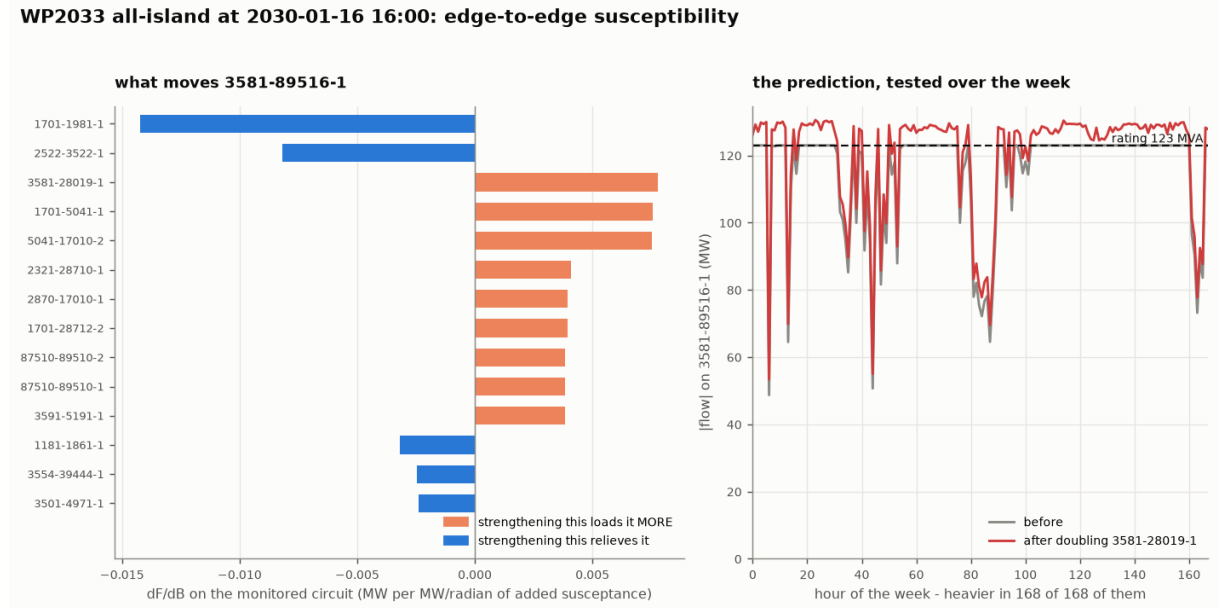


Figure 8: Left: the branches that move the island's most-loaded circuit, signed. Of the 979 other branches, strengthening **516** of them makes it worse. Right: the derivative is a claim, so the script tests it. It doubles the susceptance of the strongest candidate – what a second circuit on the same route does – holds the dispatch fixed, and re-solves the flow.

The result. Doubling 3581-28019-1 makes 3581-89516-1 carry more power in **168 of 168 hours**. Flow at the studied hour goes from 123.0 to 127.3 MW, against a 123 MVA rating. A reinforcement that a reasonable planner might propose makes the binding constraint worse.

Code: `participant-kit/flowmath.py → susceptibility(), braess_candidates()`

6.8 (f) Shift factors: the Wind Dispatch Tool calculation

For one monitored circuit, how much of each generator's output that circuit carries. Rank the generators by it and you have drawn a constraint group: the machines a dispatch instruction would have to act on.

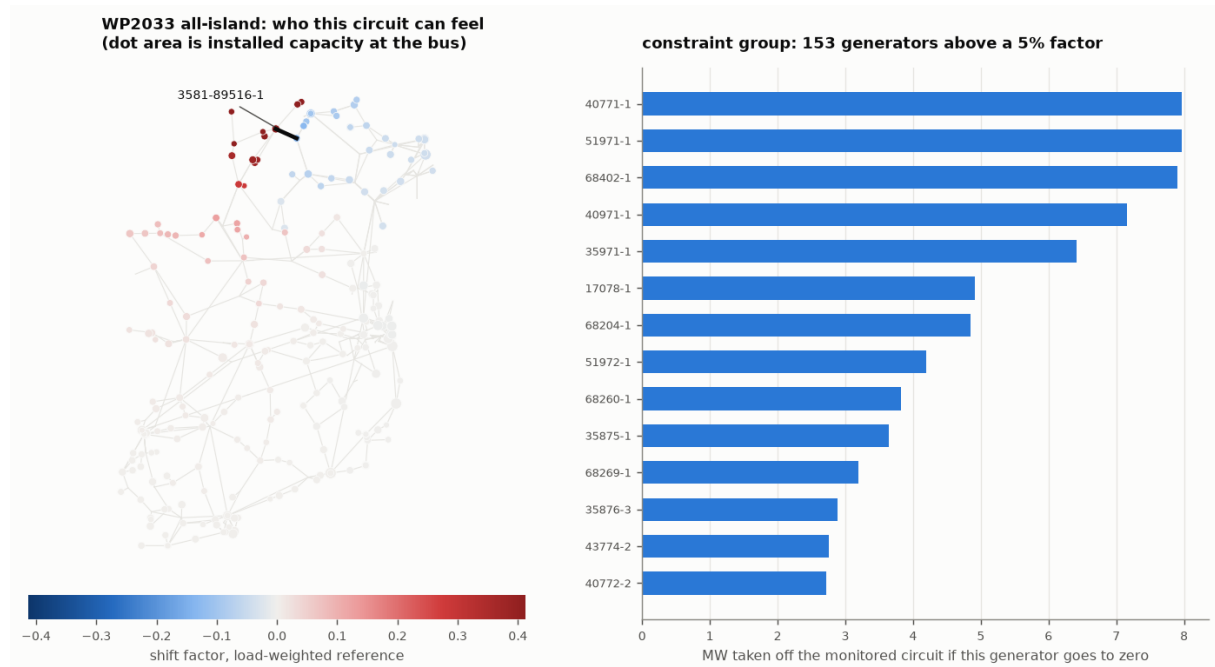


Figure 9: Left: the shift factor at every bus, dot area being installed capacity. Right: the constraint group, ranked by the MW of relief each member could actually deliver at the studied hour.

The one thing to be careful about is the *reference*. A shift factor is “MW on the circuit per MW at the generator”, but a megawatt cannot appear on its own, so the definition is incomplete until you say where the balancing megawatt goes. The kit offers three conventions:

<code>reference="load"</code>	spread over every load in proportion to its size – what a system operator means
<code>reference="uniform"</code>	spread evenly over all buses – the pseudoinverse’s own reference
a bus name	all of it at that bus – the textbook single-slack convention

The values differ; the ranking does not. Spearman rank correlation between the three references is **1.0000**. That invariance is why the method is robust enough to run a real dispatch tool on – and it is worth knowing that it is an invariance rather than a coincidence.

Code: `participant-kit/flowmath.py` → `shift_factors()`

7 Findings

7.1 Two independent methods agree on the binding circuit

This is the result worth the most, because nothing was tuned to produce it.

	Section 4	Section 6
network	2024	2033
method	DC power flow, other ties pinned at the case's own values	full LOPF over a week
profiles	none – a capacity-factor sweep	synthetic correlated weather
scope	the North-West region	the whole island
answer	3581-89516-1, Letterkenny–Strabane, 93 MVA into Northern Ireland	

Table 8: Different network vintage, different method, different data, no shared code beyond the conversion itself. Same circuit. A single method that finds a constraint has found a property of its own assumptions until something else agrees with it.

7.2 Most curtailment is not the network's fault

Section 6 again: of 291 GWh curtailed in WP2033, 265 is surplus and 26 is congestion. The instinct on seeing a large curtailment figure is to propose reinforcement. In this scenario, 91% of it would be money spent on energy that has nowhere to go regardless.

7.3 Reinforcement can make it worse, and here is a case

Section 6.7. Of the 979 branches that could be strengthened, 516 would increase the flow on the island's most-loaded circuit. The strongest candidate, verified on a re-solved power flow, makes it worse in every hour of the week. This is not a curiosity – it is a reason to compute the susceptibility matrix before proposing a circuit.

7.4 Three findings about the hand-built dataset

Restated from Section 4 because they are actionable: the hydro capacities are the 38 kV connection voltage rather than capacities; the 24-node and 15-node blocks differ by exactly Cunghill's 35 MW; and splitting Srananagh back into two nodes takes the model's flow error from 21.6 MW to 1.7.

8 Bugs found, and what each cost

Each of these produced a network that ran. That is the reason to write them down.

1. **Every LOPF infeasible.** PyPSA 1.x turns a non-null `p_set` into a hard equality constraint pinning the dispatch. The networks carried the TYTFS snapshot's own dispatch, which is the case's answer asserted – and infeasible against a lossless model. Fixed by `pypsa_net.for_optimisation()`, which clears it and keeps the case's values in a column of their own.
2. **The full network still infeasible.** The coupler-bounding rule, applied to sub-110 kV distribution transformers, produced limits that bind on the case's own flows – 13 transformers, worst at 156%. Fixed by restricting the rule to zero-impedance couplers.
3. **WP2033's angles out by 100°.** PyPSA's own slack choice survived a per-label assignment; one 21 MW hydro machine absorbed 856 MW. Section 2.6. Angle correlation $0.90 \rightarrow 0.99$.

4. **The LOPF exported 1,223 MW for profit.** A single bidirectional generator with a positive marginal cost and `p_min_pu = -1` gets *paid* to run backwards. Split into an import generator and a `sign=-1` export sink.
5. **7.8 GW of “wind” running flat out.** 82 wind and solar generators in WP2033 had no `p_max_pu` series, because the synthetic weather is generated at geocoded sites and their buses never matched OpenStreetMap. PyPSA defaults a missing series to 1.0, so at a zero bid they ran at full output for all 168 hours – more than the island’s peak demand, a fifth of the fleet, must-run baseload labelled “wind”. The curtailment figure read 64% instead of 26%. Each now borrows the nearest profile of its own carrier. **The dispatch chart looked entirely reasonable throughout.**
6. **The North-West had no wind at all.** `northwest.extract()` folds every machine at a station into one generator with carrier AC, which was right for the reconciliation it was written for. For the kit it meant a region that is 90% wind could not curtail. Now split by carrier from the case’s own machine records.
7. **Flows out by 79 MW.** The two phase-shifting transformers, Section 6. Caught only because the PTDF was checked against a power flow.
8. **A circuit at 1600% of its rating.** `lpf` after `optimize` without freezing the dispatch. Now `gridkit.freeze_dispatch()`, with the symptom described in its docstring so the next person recognises it.
9. **Ireland in the corner of a map of the Atlantic.** Three buses at 0°N 0°E. Now `gridkit.placed_buses()`.
10. **Guards that never fired.** Several `if not len(frame_t.p)` checks were testing the row count of an empty-column DataFrame – which is the snapshot count, and always truthy – so `freeze_dispatch` would silently freeze nothing.

9 Tests

```
python -m pytest # 215 passed
cd participant-kit && python test_kit.py # 20/20, no pytest needed
```

file	tests	what it guards
<code>test_network.py</code>	37	the earlier graph work
<code>test_profiles.py</code>	32	the ERA5 pipeline, and that it refuses to invent
<code>test_synthetic.py</code>	31	correlation, anchoring, determinism
<code>test_psse.py</code>	30	the raw-file parser, against the files themselves
<code>test_pypsa_net.py</code>	26	every conversion decision in Section 2
<code>test_northwest.py</code>	21	the station map, the circuit count, both views
<code>participant-kit/test_kit.py</code>	20	the kit: networks, loader, linear algebra
<code>test_geocode.py</code>	18	normalisation, NI codes, the accept/reject rules
215		

9.1 The two that matter most

Most tests check that code does what it says. These two check it against something that does not share code with it.

`test_ptdf_matches_a_power_flow` rebuilds every branch flow from PTDF $\times p$ and compares against PyPSA’s own `lpf`. Two routes to the same number, agreeing to 10^{-6} MW. This is the test that found the phase shifters.

`test_susceptibility_matches_finite_differences` perturbs one branch’s susceptance by a ten-thousandth and checks the analytic derivative against the difference it actually makes:

```
numeric = (flows_with(bumped) - reference) / step
predicted = analytic.iloc[:, column].to_numpy()
# The natural scale is MW per MW/radian. Some columns are exactly zero -
# a radial stub's susceptance changes no flow anywhere - so the column's
# own magnitude is not a usable denominator on its own.
scale = max(np.abs(predicted).max(), np.abs(reference).max() / base[column])
assert np.abs(numeric - predicted).max() / scale < 1e-4
```

If either drifts, nothing built on `flowmath.py` means anything.

Code: `participant-kit/test_kit.py` → `test_ptdf_matches_a_power_flow()`

Code: `participant-kit/test_kit.py` → `test_susceptibility_matches_finite_differences()`

10 Limitations

Read this before presenting anything from it.

This is a DC approximation of an AC model. The TYTFS files are full AC load-flow cases with reactive power, voltage magnitudes, tap changers and shunt compensation. All of that is thrown away: flows are linear in angle differences, voltages are 1.0 pu everywhere, losses are zero. That is the standard approximation for transmission constraint work and is good to a few percent on real flows, but it says nothing about voltage, reactive support or stability – and a conclusion that depends on those is not supported by anything here.

Line lengths are placeholders. The `LEN` field is zero or nominal on most branches. Impedances are real and come from the file; it is the impedances, not the lengths, that set the flows. Any per-km cost derived from these lengths is derived from a placeholder.

Ratings are TYTFS RATE1, not operational limits. A control room works to seasonal, weather-dependent and often dynamic limits that are not in these files, and runs circuits above RATE1 for defined periods. “100% of rating” here means 100% of a planning number.

Every time series is synthetic. No hour in them ever happened. They are anchored so each network’s own scenario hour reproduces the TYTFS state, and the spatial correlation is measured back off the output rather than assumed – but they are not a historical year and not a forecast. Generators without a geocoded site borrow a neighbour’s profile, which makes those pairs perfectly correlated when they should not be.

Costs are placeholders, and renewables bid negative. Gas 90, biomass 40, hydro 1, imports 150 EUR/MWh, wind and solar at -1 . The negative bid is deliberate – it is how a support scheme makes a wind farm willing to pay to stay on, and it is what makes curtailment a last resort rather than a free choice. It also makes the objective negative, so only *differences* between two objectives mean anything.

About a fifth of generation carries the carrier unknown. Carriers are inferred from bus-name prefixes and a keyword list; the files do not label plant by technology. Any result turning on the split between `gas` and `unknown` is unsupported.

The DC links are simplified. Fixed efficiency, no ramp limits, no minimum stable export, no market coupling. Interconnector flow here is what the cost assumptions make of it, not what the day-ahead market would do.

Only one circuit binds, and the far side is an infinite sink. The model prices export to Northern Ireland at nothing beyond the tie’s thermal limit. In reality SONI would redispatch around it. What the result establishes is that the profiles produce binding constraints and where – not that any particular curtailment percentage is right.

The TYTFS data is EirGrid’s, provided for reference purposes only. It describes a planning forecast, not the operational system, and is published without warranty as to accuracy or fitness for any particular purpose. Nothing here is an EirGrid product or an EirGrid position.

11 Where everything is

file	lines	what it does
<i>build half</i>		
psse.py	675	reads PSS/E v35 raw files
pypsa_net.py	1352	the conversion, Section 2
geocode.py	1085	OpenStreetMap matching, Section 3
northwest.py	958	the region, Section 4
profiles.py	948	the ERA5 pipeline (never run – blocked host)
synthetic.py	964	the synthetic generator, Section 5
build_kit.py	459	assembles the participant kit
<i>participant half – imports none of the above</i>		
participant-kit/gridkit.py	345	the loader
participant-kit/flowmath.py	340	PTDF, susceptibility, shift factors
participant-kit/plotstyle.py	99	the palette
participant-kit/test_kit.py	300	the checks
participant-kit/examples/	—	six scripts, a to f
<i>this document</i>		
docs/report_figures.py	276	figures 1, 2 and 4
docs/report/report.tex	—	the source of this report
<i>the long-form working notes</i>		
docs/PHASE2_PYPSA.md	605	every conversion decision, with its evidence
docs/PHASE3_NORTHWEST.md	474	the reconciliation, table by table
docs/PHASE4_PROFILES.md	338	the ERA5 pipeline and the blocker
docs/SYNTHETIC_DATA.md	333	what is real and what is fabricated

Table 9: This report is the summary. The phase documents are the working notes, and where a number here has a longer story, that is where it is.

Reproducing every figure in this document.

```
python docs/report_figures.py           # figures 1, 2, 4
cd participant-kit
for e in examples/*.py; do python "$e"; done # figures 3, 5-9
cd ../docs/report && pdflatex report.tex
```